
Chapter 5: Communications Via Network and the Web



Content

- Introduction
- Communication, network and web
- Telephony
- Notifications and Alarms

1. Introduction

- **Network communication**, or **inter-networking**, defines a set of **protocols** (rules and standards) that allow application programs to talk to each other without regard to the hardware and **operating systems** where they are running.
- **Inter-networking** allows application programs to communicate **independent** of their physical connections.
- The **Inter-networking** technology called **TCP/IP** is named after its two main protocols:
 - **Transmission Control Protocol (TCP)**: A standard defining how to establish and maintain a network conversation by which applications can exchange their data.
 - **Internet Protocol (IP)**: A component performing a routing function in data communication.

Cont'd...

- To understand TCP/IP, You should be familiar with the following terms:
 - **Client**: A component/node that requests services on the network.
 - **Server**: A component usually a host that responds to a request for service from a client.
- Computer networking began in the 1960s.
- In 1990s, when Internet came into existence, Internet technologies began to pervade and expand their services in all fields that they were involved.
- Now, with the Internet as a global phenomenon, it is unthinkable to have a computer that don't include communications capabilities.
- Any computing machine is almost useless without network and communication capability this days.

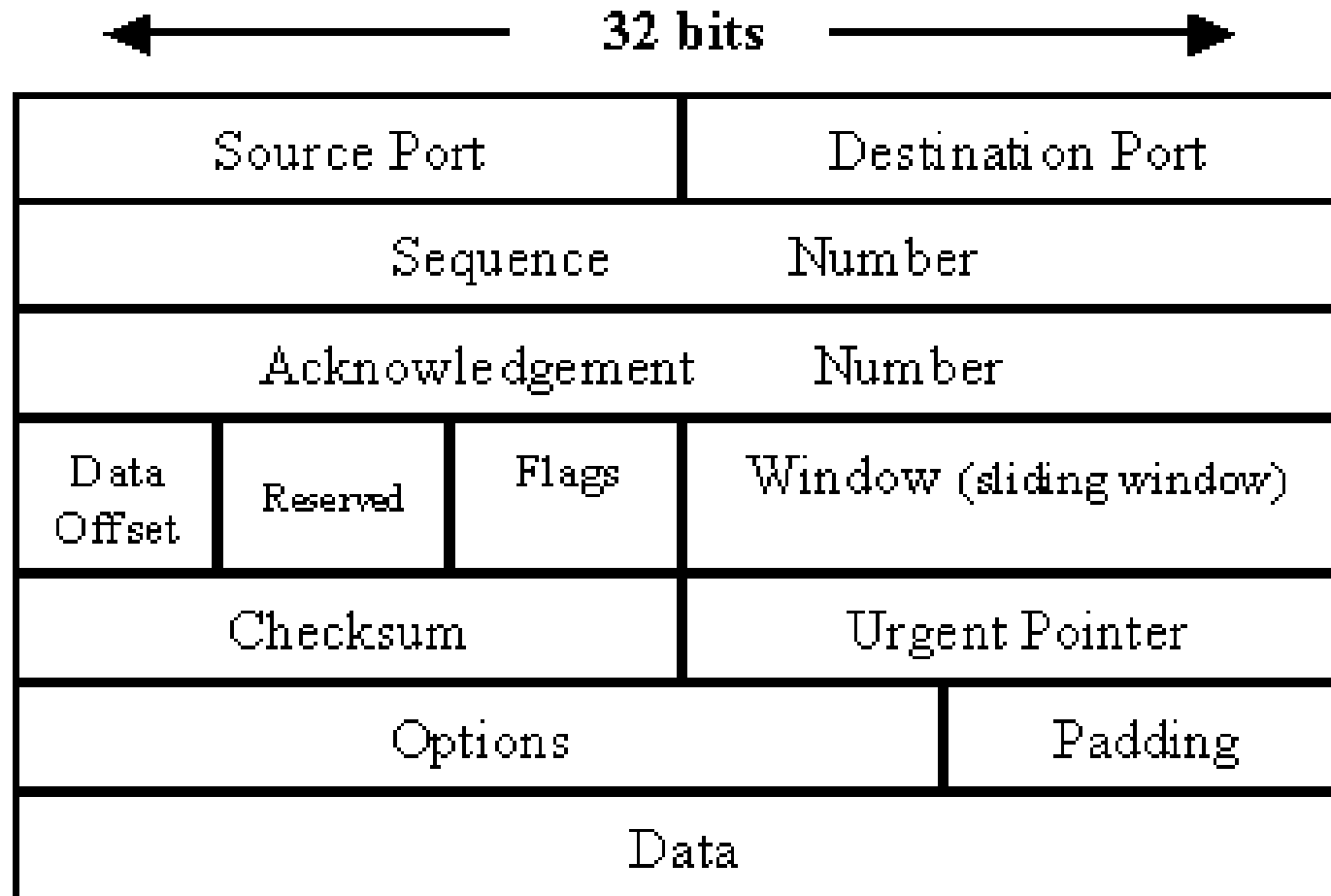
2. Communications Via Network and the Web

- As ARPANET(Advanced Research Projects Agency Network)grew and gained popularity, other networks also came into existence and each had needed to be connected to one another through networks to perform their day to day activities.
- This led to a problem:→ The networks could not talk to each other.
- Each network used its own **proprietary language, or protocol**, to send information back and forth.
- This problem was solved by the invention of **Transmission control protocol/Internet protocol (TCP/IP)**.
- TCP/IP was designed to allow networks running on **different protocols** to have an **intermediary protocol** that would allow them to communicate well.
- So as long as your network supports **TCP/IP**, you could communicate with all of the other networks running **TCP/IP**.
- TCP/IP quickly became the **standard protocol** and allowed networks to **communicate** with each other.

Cont'd...

- It is from this breakthrough that we first got the term **Internet**, which simply means **an interconnected network of networks**.
- Basic terms in networking and communication
- **Packet**: The fundamental unit of data transmitted over the Internet.
- When a device needs to send a message to another device (for example, your PC sends a request to YouTube to open a video) → It breaks the message down into smaller pieces, called packets.
- Each packet has:
 - The sender's address,
 - The destination address,
 - A sequence number, and
 - A piece of the overall message/data to be sent.

Cont'd...



Network Devices

- **Hub**: a simple network device that connects devices to the network and sends packets to all the devices connected to it.
- **Switch**: a network device that connects multiple devices/computers together and filters(send) packets based on their destination address.
 - It identify devices using their MAC address.
- **Router**: a device that receives & analyzes packets and then routes them towards their destination network using the IP address.
 - Router see two networks as a discrete entity.
 - It identify the connected devices with their IP address.

Connecting to the Network

- To perform network operations in our application, our manifest must include the following permissions:

```
<uses-permission android:name="android.permission.INTERNET" />
```

```
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
```

- Before we add networking functionality to our app, we need to ensure that data and information within our app stay safe when you are transmitting over a network.
- To do so, follow these networking security best practices:
 - ✓ Minimize the amount of sensitive or personal [user data](#) that you transmit over the network.
 - ✓ Send all network traffic from your app over [SSL](#).
 - ✓ Consider creating a [network security configuration](#), which lets your app trust custom certificate authorities (CAs) or restrict the set of system CAs that it trusts for secure communication.

Check the Network Connection

- Before we attempts to connect to the network, it should check to see whether a network connection is available using **getActiveNetworkInfo()** and **isConnected()**.
- Remember, the device may be out of range of a network, or the user may have disabled both Wi-Fi and mobile data access.

```
ConnectivityManager connMgr = (ConnectivityManager) context.getSystemService(context.CONNECTIVITY_SERVICE);  
NetworkInfo networkInfo = connMgr.getActiveNetworkInfo();  
If(networkInfo!=null && networkInfo.isConnected){  
    //Fetch data}  
else{  
    //Display Error}
```

Android Networking and Web: Checking internet connectivity

```
protected void onCreate(Bundle savedInstanceState) {  
    super.onCreate(savedInstanceState);  
    setContentView(R.layout.activity_main);  
    Button check=findViewById(R.id.checkConn);  
    check.setOnClickListener(new View.OnClickListener() {  
        @Override  
        public void onClick(View v) {  
            if(isConnected()){  
                Toast.makeText(getApplicationContext(), text: "Internet Connection is Available", Toast.LENGTH_LONG).show();  
            }  
            else  
                Toast.makeText(getApplicationContext(), text: "Internet Connection is Not Available", Toast.LENGTH_LONG).show();  
        }  
    });  
}  
  
boolean isConnected(){  
    ConnectivityManager manager=(ConnectivityManager) getSystemService(Context.CONNECTIVITY_SERVICE);  
    NetworkInfo netInfo=manager.getActiveNetworkInfo();  
    if(netInfo!=null){  
        if (netInfo.isConnected()){  
            return true;  
        }  
        else return false;  
    }  
    return false;  
}
```

Network Operations on a separate Thread

- Network operations can involve unpredictable delays. To prevent this from causing a poor user experience, always perform network operations on a separate thread from the UI.
- The **AsyncTask** class provides one of the simplest ways to fire off a new task from the UI thread.

```
private class DownloadWebpageTask extends AsyncTask<String, void, String>{  
    @Override  
    Protected String doInBackground(String ...URLs){  
        try{  
        }catch(IOException e){  
            return "Unable to retrieve web page. URL may be invalid.";  
        }  
    }  
    @Override  
    Protected void onPostExecute(String result){  
        textView.setText(result);  
    }  
}
```

Managing Network Usage

- It is important to know how to write applications that have fine-grained control over their usage of network resources.
- If your application performs a lot of network operations, you should provide user settings that allows users to control your app's data habits, such as how often your app syncs data, whether to perform uploads/downloads only when on Wi-Fi, whether to use data while roaming, and so on.
- With these controls available to them, users are much less likely to disable your app's access to background data when they approach their limits, because they can instead precisely control how much data your app uses.

Check a device's Network Connection

- A device can have various types of network connections. This lesson focuses on using either a Wi-Fi or a mobile network connection. For the full list of possible network types, see **ConnectivityManager**.
- Wi-Fi is typically faster. Also, Mobile data is often metered, which can get expensive. A common strategy for apps is to only fetch large data if a Wi-Fi network is available.
- Before you perform network operations, it's good practice to check the state of network connectivity.
- Among other things, this could prevent your app from inadvertently using wrong radio.
- If a network connection is unavailable, your application should respond gracefully.

Network Connection Type Example

- To check the network connection, you typically use the following classes:
 - **ConnectivityManager:** answers queries about the state of network connectivity. It also notifies applications when network connectivity changes.
 - **NetworkInfo:** Describes the status of a network interface of a given type (currently either mobile or Wi-Fi).

```
private static final String DEBUG_TAG = "NetworkStatusExample";  
ConnectivityManager connMgr = (ConnectivityManager) getSystemService(Context.CONNECTIVITY_SERVICE);
```

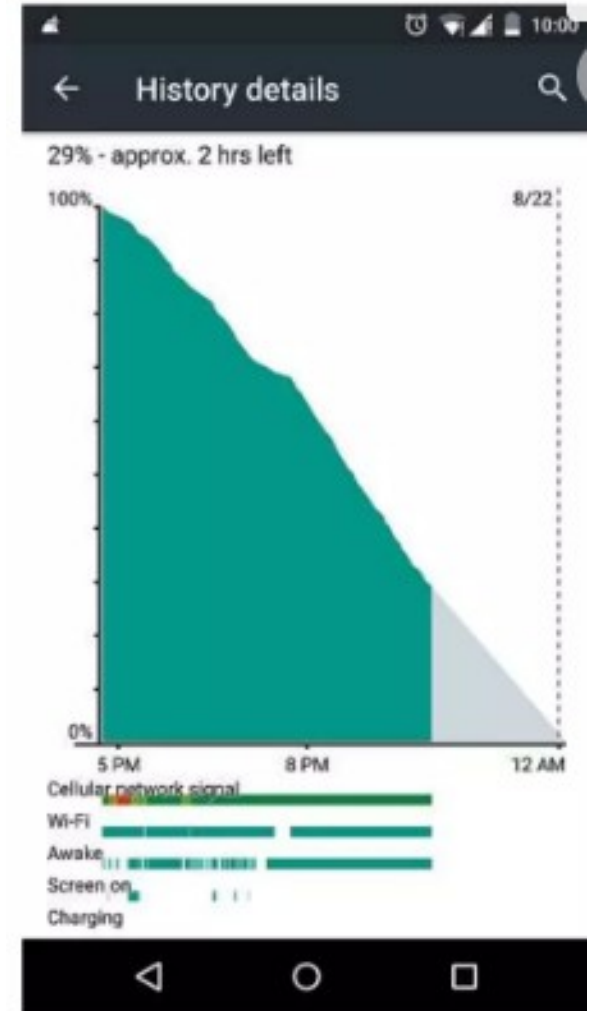
```
NetworkInfo networkInfo = connMgr.getNetworkInfo(ConnectivityManager.TYPE_WIFI);  
Boolean isWifiConnected = networkInfo.isConnected();
```

```
networkInfo = connMgr.getNetworkInfo(ConnectivityManager.TYPE_MOBILE);  
Boolean isMobileConnected = networkInfo.isConnected();
```

```
Log.d(DEBUG_TAG, "Wifi connected:" + isWifiConnected);  
Log.d(DEBUG_TAG, "Mobile connected:" + isMobileConnected);
```

Efficient Network Access

- Using the wireless radio to transfer data is potentially one of your app's most significant sources of battery drain.
- To minimize the battery drain associated with network activity, it's critical that you understand how your connectivity model will affect the underlying radio hardware.
- It is good approach to minimize your data connections, use prefetching, and bundle your transfers in order to minimize the battery drain associated with your data transfers.



Android Libs For Effective Networking

- Just a small amount of them
 - Retrofit
 - okhttp
 - Volley
 - RoboSpice

Performance & Memory Management

- Performance testing checks the total effectiveness of an application in relation to its throughput, response time effectiveness and the availability.
- Perceived performance issues
 - The time taken from startup to get rendered.
 - The time it takes for a network call to get executed.
 - The number of network calls made to get the required data.
 - How easily the events are captured in the application?
- Memory management requires that the programmer provides ways to dynamically allocate portions of memory to programs when requested, and free it for reuse when it is no longer needed.
- In any Advanced Computer System, where more than a single process might be running at any given point in time, performance & memory management is critical.

Bluetooth

- Android platform includes support for the Bluetooth network stack, which allows a device to wirelessly exchange data with other devices. The application framework provides to the Bluetooth functionality through the android Bluetooth APIs. These APIs let applications wirelessly connect to other Bluetooth devices, enabling point-to-point and multipoint wireless features.
- Using the Bluetooth APIs, android application can perform the following:
 - Scan for other Bluetooth devices
 - Query the local Bluetooth adapter for paired Bluetooth devices
 - Establish RFCOMM channels
 - Connect to other devices through service discovery
 - Transfer data to and from other devices
 - Manage multiple connections

Bluetooth Permissions

- In order to use Bluetooth features in your application, you must declare the Bluetooth permission **Bluetooth**. You need this permission to perform any Bluetooth communication.
- If you want your app to initiate device discovery or manipulate Bluetooth settings, you must also declare the **BLUETOOTH_ADMIN** permission.
- Most applications need this permission solely for the ability to discover local Bluetooth devices.
- The other abilities granted by this permission should not be used, unless the application is a “power manager” that will modify Bluetooth settings upon user request.
- If you use **BLUETOOTH_ADMIN** permission, then you must also have the **BLUETOOTH** permission.

<uses-permission android:name=“android.permission.BLUETOOTH”/>

Beginning work with Bluetooth

- *Get the BluetoothAdapter*

BluetoothAdapter mBluetoothAdapter = BluetoothAdapter.getDefaultAdapter();

If(mBluetoothAdapter==null){//Device does not support Bluetooth}

Enable Bluetooth if not enabled

If(mBluetoothAdapter.isEnabled){

Intent enableBluetoothIntent = new Intent (BluetoothAdapter.ACTION_REQUEST_ENABLE)

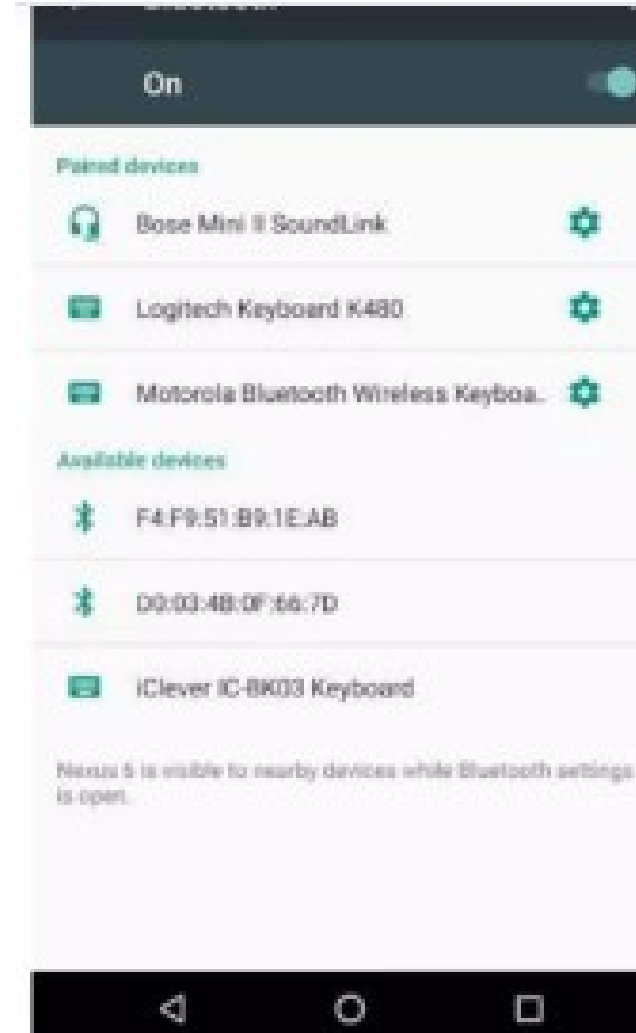
startActivityForResult(enableBluetoothIntent, REQUEST_ENABLE_BT);

}

- Optionally, your application can also listen for the ACTION_REQUEST_CHANGED broadcast intent, which the system will broadcast whenever the Bluetooth state has changed. This broadcast contains the extra fields EXTR_STATE and EXTRA_PREVIOUS_STATE, containing the new and old Bluetooth states, respectively. Possible values for these for extra fields are STATE_TURNING_ON, STATE_TURNING_OFF, and STATE_OFF.

Discovery

- Dive Discovery is a scanning procedure that searches the local area for Bluetooth enabled devices and then requesting some information about each one (this is sometimes referred to as “discovering” ,”inquiring ” or “Scanning”).
- However, Bluetooth device within the local area will respond to a discovery request only if it is currently enabled to be discoverable.
- If a device is discoverable, it will respond to the discoverable, it will respond to the discover request by sharing some information, such as the device name, class, and its unique MAC address.
- Using this information, the device performing discovery can the choose to initiate a connection to the discovered device.



Querying paired devices

- Before performing device discovery, its worth querying the set of paired devices to see if the desired device is already known.
- To do so , call `getBondedDevices()`. This will return a set of Bluetooth Devices representing paired devices.
- For example, you can query all paired devices and then show the name of each device to the user:

```
Set<BluetoothDevice> pairedDevices = mBluetoothAdapter.getBondedDevices();  
If(pairedDevices.size()>0){  
    //Loop through paired devices  
    For(BluetoothDevice device: pairedDevices){  
        // Just output Name and address to Log  
        Log.d("Devices Info", device.getName()+"\n"+device.getAddress())  
    }  
}
```

Connecting to Devices

- When you want to connect two devices, one must act as a server by holding an open *BluetoothServerSocket*.
- The purpose of the server socket is to listen for incoming connection requests and when one is accepted, provide a connected *BluetoothSocket*.
- When **BluetoothSocket** is acquired from the **BluetoothServerSocket**, the **BluetoothServerSocket** can (and should) be discarded, unless you want to accept more connections.
 - Get a **BluetoothServerSocket** by calling the **ListenUsingRfcommWithServiceRecord(String, UUID)**.
 - Start listening for connection requests by calling **accept()**.
 - Unless you want to accept additional connections, call **close()**.

Bluetooth Profiles

- You can implement the interface **BluetoothProfile** to write your own classes to support a particular Bluetooth profile.
- The android Bluetooth API provides implementations for following Bluetooth profiles:
 - **Headset.** The Headset profile provides support for Bluetooth headsets to be used with mobile phones.
 - **A2DP.** The Advanced Audio Distribution profile(A2DP) profile defines how high quality audio can be streamed from one device to another over a Bluetooth connection.
 - **Health Device.** Android 4.0 (API level 14) Introduces support for the Bluetooth Health device profile(HDP).
 - This lets you create applications that use Bluetooth to communicate with health devices that support Bluetooth, Such as heart-rate monitors, blood meters, thermometers, scales, and so on.

Near Field Communication

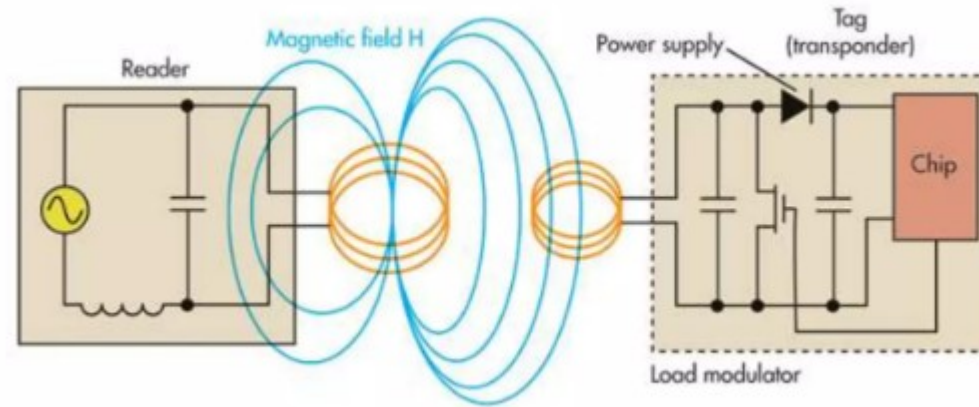
- Near Field communication(NFC) is a set of short-range wireless technologies, typically requiring a distance of 4cm or less to initiate a connection. NFC allows you to share small payloads of data between an NFC tag and an android-powered device, or between two android-powered devices.
- Tags can range in complexity. Simple tags offer just read and write semantics, sometimes with one-time-programmable areas to make the card read-only. More complex tags offer math operations, and have cryptographic hardware to authenticate access to a sector.
- The most sophisticated tags contain operating environments, allowing complex interactions with code executing on the tag. The data stored in the tag can also be written in a variety of formats, but many of the android framework APIs are based around a NFC Forum standard called NDEF (NFC Data Exchange Format).

NFC Modes

- Android-powered devices with NFC simultaneously support three main modes of operation:
 - Reader/writer mode, allowing the NFC device to read and /or write passive NFC tags and stickers.
 - P2p Mode, allowing the NFC device to exchange data with other NFC peers; this operation mode is used by android Beam.
 - Card emulation mode, allowing the NFC device itself to act as an NFC card. The emulated NFC card can then be accessed by an external NFC reader, such as an NFC point-of-sale terminal.

NFC: is it all?

- NFC is a technology which is complex enough.
- NFC communication performs through magnetic field coupling, like RFID.



Android Telephony

- A Telephony application in android is responsible for making voice communication through the telephone network.
- It also provides information about the device and its details, including the IMEI and network type.
- In addition, Android telephony provides information about SMS.
- Telephony is the technology that lets us to communicate with others over long distances.
- Previously, Telephony used to mean speaking on an analog telephone, But with the advent of digital phones, It is expanded to include voicemail and video conferencing even with multiple people.
- The Android telephony API allows developers to make outbound calls, exchange SMS and modify existing ones.

Device Information

```
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.main_telephony_info);
    textView1 = (TextView) findViewById(R.id.textView1);
    TelephonyManager tm = (TelephonyManager) getSystemService(Context.TELEPHONY_SERVICE);
    String IMEINumber = tm.getDeviceId();
    String subscriberID = tm.getSubscriberId();
    String SIMSerialNumber = tm.getSimSerialNumber();
    String networkCountryISO = tm.getNetworkCountryIso();
    String SIMCountryIOS = tm.getSimCountryIso();
    String strPhoneType = "";
    int phoneType = tm.getPhoneType();
    switch (phoneType){
        case(TelephonyManager.PHONE_TYPE_CDMA):
            strPhoneType="CDMA";
            break;
        case(TelephonyManager.PHONE_TYPE_GSM):
            strPhoneType="GSM";
            break;
        case (TelephonyManager.PHONE_TYPE_NONE):
            strPhoneType="NONE";
            break;
    }
    boolean isRoamming = tm.isNetworkRoaming();
    String Info = "Phone Details:";
    Info += "\nIMEINumber:"+IMEINumber;
    Info += "\nsubscriberID:"+subscriberID;
    Info += "\nSIMSerialNumber:"+SIMSerialNumber;
    Info += "\nnetworkCountryISO:"+networkCountryISO;
    Info += "\nSIMCountryIOS:"+SIMCountryIOS;
    Info += "\nstrPhoneType:"+strPhoneType;
    textView1.setText(Info);
}
```

```

public class NetworkClass extends Activity {
    Button Call, SMS;
    EditText txtSMS, txtPhone;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.main_telephony);
        txtSMS = findViewById(R.id.txtSMS);
        txtPhone = findViewById(R.id.txtPhone);
        Call = findViewById(R.id.btnCall);
        SMS = findViewById(R.id.btnSMS);
        Call.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                String number = txtPhone.getText().toString();
                Intent callIntent = new Intent(Intent.ACTION_DIAL, Uri.parse("tel:"+number));
                startActivity(callIntent);
            }
        });
        SMS.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                String number = txtPhone.getText().toString();
                String msg = txtSMS.getText().toString();
                ActivityCompat.requestPermissions(NetworkClass.this, new String[]{
                    Manifest.permission.SEND_SMS, Manifest.permission.READ_SMS},
                PackageManager.PERMISSION_GRANTED);
                SmsManager smsManager = SmsManager.getDefault();
                smsManager.sendTextMessage(number, null, msg, null, null);
            }
        });
    }
}

```

Notifications and Alarms

- Notifications are handy ways to add features and functionality to your application **without your users directly interacting with them.**
- For example, they can be used
 - To add playback controls to a music app,
 - Provide quick actions to respond to
 - SMS,
 - Email and alert about different breaking news and stories among many others.

Notification example

```
protected void onCreate(Bundle savedInstanceState) {  
    super.onCreate(savedInstanceState);  
    setContentView(R.layout.activity_main);  
    Button createNotification=findViewById(R.id.button);  
    if(Build.VERSION.SDK_INT >=Build.VERSION_CODES.P){  
        NotificationChannel channel=new NotificationChannel( id: "NotificationExample", name: "My Notification Example",  
            NotificationManager.IMPORTANCE_HIGH);  
        NotificationManager manager=getSystemService(NotificationManager.class);  
        manager.createNotificationChannel(channel);  
    }  
    createNotification.setOnClickListener(new View.OnClickListener() {  
        @Override  
        public void onClick(View v) {  
            NotificationCompat.Builder builder=new NotificationCompat.Builder( context: MainActivity.this,  
                channelId: "NotificationExample");  
            builder.setContentTitle("My Notification Title");  
            builder.setContentText("Practice more to be proficient in programming");  
            builder.setSmallIcon(R.drawable.ic_launcher_background);  
            builder.setAutoCancel(true);  
            NotificationManagerCompat managerCompat=NotificationManagerCompat.from(MainActivity.this);  
            managerCompat.notify( id: 10, builder.build());  
        }  
    });  
}
```

Alarm

- Alarms which are based on [AlarmManager](#) and [BroadcastReceiver](#) classes give us a way to perform time-based operations outside the lifetime of our application.
- Android AlarmManager allows us to access system alarm.

```
public void setBtn(View view){
    Calendar calendar=Calendar.getInstance();
    calendar.set(calendar.get(Calendar.YEAR), calendar.get(Calendar.MONTH),
        calendar.get(Calendar.DAY_OF_MONTH), timePicker.getHour(),
        timePicker.getMinute(), second: 0);
    alarmSet((calendar.getTimeInMillis()));
}

private void alarmSet(long timeInMillis){
    AlarmManager alarmManager=(AlarmManager)getSystemService(Context.ALARM_SERVICE);
    Intent intent=new Intent( packageContext: this, Alarm.class);
    PendingIntent pendingIntent=PendingIntent.getBroadcast( context: this, requestCode: 0,intent, flags: 0);
    alarmManager.setRepeating(AlarmManager.RTC_WAKEUP, timeInMillis,AlarmManager.INTERVAL_DAY,pendingIntent);
    Toast.makeText( context: this, text: "Alarm Set", Toast.LENGTH_LONG).show();
}
```

Cont'd...

```
public class Alarm extends BroadcastReceiver {  
    @Override  
    public void onReceive(Context context, Intent intent) {  
        MediaPlayer player=MediaPlayer.create(context, Settings.System.DEFAULT_ALARM_ALERT_URI);  
        player.start();  
        Toast.makeText(context, text: "Player Started", Toast.LENGTH_SHORT).show();  
    }  
}
```

Cont'd...

```
<application
    android:allowBackup="true"
    android:dataExtractionRules="@xml/data_extraction_rules"
    android:fullBackupContent="@xml/backup_rules"
    android:icon="@mipmap/ic_launcher"
    android:label="Alarm"
    android:roundIcon="@mipmap/ic_launcher_round"
    android:supportsRtl="true"
    android:theme="@style/Theme.Alarm"
    tools:targetApi="31">
    <activity
        android:name=".MainActivity"
        android:exported="true">
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />
            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>
    <receiver android:name=".Alarm"
        android:exported="true"
        android:enabled="true"/>
</application>
```

...Cont'd

The End!